

Staff Engineer (Takehome)

Code-Execution Inside Our AI Chat

Overview

Welcome to our technical challenge for the **Staff Engineer** position. This challenge is designed to evaluate your ability to own a complex problem end-to-end: from system design and architectural decision-making to hands-on prototyping and stakeholder communication.

Format: This is a design, architecture, and prototyping challenge. You will deliver a working proof-of-concept, a written Architecture Decision Record (ADR), and a video walkthrough. See **Delivery** at the end for full details.

The Business Problem

We are a fast-moving legal-tech start-up. Our AI-powered chat product lets lawyers ask questions about their caseload in natural language. User questions have become increasingly sophisticated — they no longer stop at *"Summarise last quarter's cases."* Instead, they ask things like:

"Compare the amount of Real Estate cases closed over the last three months against the same period last year."

"Pull the case from client X from three months ago and calculate our provision." (The system knows that the provision is 30% over the fine to be paid by the plaintiff.)

Our current large-language-model integration handles prose brilliantly but collapses when faced with precise maths, date arithmetic, or ad-hoc file parsing. Several pain points surface:

1. **Inconsistent date reasoning** — The LLM interprets "three months ago" in creative but unreliable ways.

2. **No computational ability** — Even when the date range is correct, the model cannot run the decimal maths needed to validate provisions retrieved from several documents in our Vector DB.
3. **Privacy and cost concerns** — The fallback of offloading every raw record to the model is slow, expensive, and sometimes prohibited by privacy rules.

The longer we leave this gap, the more user trust erodes. Leadership has asked us to introduce a **secure, server-side code-execution pathway** that the chat agent can invoke whenever it determines that pure language generation won't cut it.

The Scenario

Imagine your first morning at the company. You receive this mandate from the Head of Engineering:

"Ship a first version of code execution as soon as possible so we can invite a private beta group to start testing. Do it largely yourself since everyone else is occupied with current commitments. Keep the code quality bar high and protect user privacy from day one."

You join as our first **Staff Engineer** on a newly formed squad. The brief is stark:

- Go from zero to an **internal demo** that proves we can safely run LLM-generated Python snippets and stream results back to the user — as fast as possible.
- **Largely solo delivery** — you are the main contributor. However, you have access to the developers who originally built the chat product.

Outside your squad you can rely on:

- A **Product Manager** who can spare one hour a day for clarifications and periodic demos.
- The **AI Lead**, who keeps one hour free each afternoon for pair-thinking.
- The **Head of Engineering**, who can support you on people and project matters.

Why this bar?

We are hiring someone who can own a problem end-to-end, negotiate ambiguity, and still land a scoped, secure solution on time. This challenge is meant to expose how you break a large problem into digestible pieces, how you communicate trade-offs, and how you defend your design when challenged.

The Technical Challenge

What Already Exists

We already have a working AI chat product. The existing stack includes:

- **Django** (Python) as the application framework
- **PostgreSQL** with **pgvector** for vector search and document retrieval
- An LLM integration that handles conversational queries, document summarisation, and natural-language search across our legal case database
- Standard cloud infrastructure (assume AWS or GCP — your choice)

The chat works well for qualitative questions. Where it breaks down is anything that requires **deterministic computation** — maths, date logic, aggregations, or structured data manipulation.

What We Need: A Code-Execution Layer

We need a secure code-execution pathway that the chat agent can invoke when it recognises that pure language generation won't produce a reliable answer. The core loop is:

1. The chat agent recognises that a user query requires computation.
2. The LLM drafts a short, self-contained Python routine.
3. We execute that code in a **sandboxed environment** with strict resource limits.
4. We capture the output (numeric result, table, or generated artifact).
5. We weave the answer back into the conversation — all within the user's attention span.

Here are concrete examples of queries that should trigger code execution:

- *"What's the total value of claims filed in Q1 this year vs. Q1 last year?"* → requires date filtering and summing amounts across records.
- *"Calculate the provision for case #4821 — the fine is €45,000."* → requires applying the 30% rule: $€45,000 \times 0.30 = €13,500$.
- *"Show me a breakdown of case outcomes by category for the last 6 months."* → requires grouping, counting, and potentially formatting a table.
- *"How many days between the filing date and the hearing date for each open case?"* → requires date arithmetic across multiple records.

Your Task: Build a POC

Build a proof-of-concept that demonstrates this code-execution loop working end-to-end.

Important: Your focus should be on the **code-execution implementation** — the sandbox, safety guardrails, and result capture — not on building the chat interface. For the chat layer, feel free to use existing scaffolding such as [Streamlit's conversational app tutorial](#) or any similar base. We want your time and effort invested in the hard problem, not in UI boilerplate.

Your POC should demonstrate:

- The sandbox execution loop (receive code → run safely → return result)
- Integration with a chat agent flow (the chat layer can be simple or borrowed from existing templates)
- Key safety guardrails in action (what happens when code tries something it shouldn't?)
- How the agent decides when to generate code vs. answer directly

You are free to introduce additional technologies where justified, but your choices should support **rapid iteration now and scalability later**.

Core Areas to Address

Your solution should thoughtfully cover the following:

1. **Sandbox Design** — How do you isolate untrusted, LLM-generated code? What are the resource limits, timeouts, and failure modes?

2. **Agent ↔ Sandbox Integration** — How does the chat agent decide when to invoke code execution? How are inputs provided and outputs captured?
3. **Security & Privacy** — How do you prevent code injection, data exfiltration, and ensure user data never leaks between sessions?
4. **Streaming & UX** — How do you keep the user informed while code is executing? What happens when execution fails or times out?
5. **Observability** — How do you log, monitor, and debug executions in production?
6. **Iteration Path** — How does this architecture evolve from a beta prototype to a production-grade system?

Architecture Decision Record (ADR)

Alongside your POC, write an ADR for the code-execution feature. This should read like an internal document you would share with the engineering team. Cover:

- **Context** — The problem and constraints
 - **Decision** — Your chosen approach and key technology choices
 - **Consequences** — Trade-offs, risks, and what you are deliberately deferring
 - **Alternatives Considered** — What else you evaluated and why you ruled it out
-

Part 2: Technical Deep-Dive Questions

Address these questions in your video walkthrough. We expect your answers to reference your actual POC code and your past professional experience. Generic or purely theoretical answers will not score well — we want to understand how you think based on what you've built.

Question 1: Sandbox Security — Show Your Work

- In your video, walk us through your sandbox isolation approach while showing the relevant code. What isolation strategy did you choose, what did you consider first, and what did you change along the way? Why?
- Run your POC on camera and demonstrate a scenario where the sandbox correctly blocks a dangerous operation (e.g., a network call, file system

access, or resource exhaustion attempt). Explain the mechanism that stops it.

- From your own experience: describe a past project where you had to secure a system against untrusted input or code execution. What surprised you? What would you not do again?

Question 2: Scope and Trade-offs Under Pressure

You have very limited time and are largely working solo. The Head of Engineering wants a working beta, not a perfect system.

- In your video, show us what you deliberately left out of your POC and explain why. For each shortcut, walk us through the risk it introduces and how you would mitigate it in a production system.
- From your own experience: tell us about a time you had to ship something under heavy time pressure while maintaining quality and security. What was the hardest trade-off, and what did you learn from how it played out?
- Prioritisation scenario: imagine the Head of Engineering asks you mid-sprint to also support JavaScript execution alongside Python. Walk us through on camera exactly how you would respond — what questions would you ask, what would you push back on, and how would you communicate the impact to stakeholders?

Question 3: When Things Go Wrong

In production, LLM-generated code will sometimes be syntactically invalid, produce wrong results, run forever, or crash the sandbox.

- Run a failure case on camera: feed your POC a prompt that causes the LLM to generate broken or problematic code. Before running it, predict on camera what will happen and why. Then run it and explain the gap between your prediction and the actual result.
- From your own experience: describe a production incident or system failure you were personally involved in — ideally related to AI/ML or infrastructure. What broke, how did you find out, what did you do in the first hour, and what did you change afterward?

- Live debugging scenario: imagine a user reports that the code-execution feature returned a silently wrong calculation (e.g., the provision amount is off by an order of magnitude). Walk us through on camera exactly how you would debug that using your POC — from alert to root cause. What observability or logging would you need that your POC doesn't have yet?
-

How We Will Assess You

Technical Design & Execution

- Depth of your sandbox architecture and system design
- A working POC that demonstrates your thinking, not just describes it
- Understanding of security, isolation, and privacy constraints
- Thoughtfulness around failure modes, edge cases, and system reliability
- Ability to make pragmatic trade-offs between ideal design and shippable MVP

Leadership & Communication

- How you communicate architectural decisions and trade-offs (the ADR)
 - How you pitch a technical vision to engineering leadership (the video)
 - How you prioritise scope under pressure and articulate what you are deferring and why
 - How you respond to scope changes and stakeholder pressure
 - How you balance hands-on technical work with strategic thinking
-

Delivery

This is an offline assessment that you can complete as a short homework and share back with us.

What to Submit

1. **Your code** (repo or zipped project) — the working POC described above. Include a README with setup instructions.

2. **An Architecture Decision Record** (as a document in the repo or a separate file) — covering your design decisions, trade-offs, and alternatives considered.
3. **A video walkthrough** (tip: using loom.com is a great option) that includes:
 - **Leadership pitch:** present the feature as you would to engineering leadership to get buy-in. Reference your prototype as evidence that the approach works and walk through the ADR to explain your reasoning. This is not a code review — it is a pitch.
 - **Technical deep-dive:** address the three questions from Part 2. Show your POC running on screen, reference your code, and draw on your past professional experience.

Timeline & Next Steps

This assessment is intended to be quick and short nowadays with AI code assistants. If you think you'll need more than one week to complete it, please let us know.

Once you send it back, we will review your submission and get back to you with the next steps. You will always receive feedback on your submission, regardless of the outcome.

If your submission passes our review, we will invite you to a follow-up session where we will go through your solutions together — your code, your video, and your ADR. Expect us to ask deeper questions about your decisions, explore alternative approaches, and discuss how you would handle scope changes or unexpected constraints.

Final Thoughts

This exercise mirrors genuine constraints you will face here: compressed timelines, ambiguous requirements, and high expectations for quality and security. We are excited to see not only what you would build, but how you reason about the problem, communicate your approach, and make pragmatic decisions under pressure.

Assume what you need to for now — we understand that in reality there would be kick-offs, discovery sessions, and many discussions beforehand. This is a hypothetical exercise, and we value your reasoning more than your assumptions. Good luck — we look forward to reviewing your work.